LLDD FE - Application Foundation and Shared UI

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน
และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	FE
Estimate	35 ชั่วโมง = implementation 28 + unit test 7 (25%)
Owner	Chidchanok <lin> Saengamnat
Target repository	`SBP/srm-sps-spsap-web-frontend` (sbp-portal · Next.js · `NEXT_PUBLIC_APP_TARGET=sbpm`) — เรียก API ผ่าน `SBP/srm-sps-spsap-sbp-bff` เท่านั้น ห้ามยิง store-backend ตรง
Objective	เตรียม foundation ฝั่ง Frontend สำหรับ SBP Mall: routing, API client, constants, shared state, formatters, mock mapping และ shared UI primitives; เอกสารนี้ไม่ใช่หน้าจอ Dashboard

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

1.1 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
ใช้ endpoint ของ	LLDD-BE-API-Lookup.pdf	`GET /api/v1/sgi/lookup/document-statuses`
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
สัญญากลาง	LLDD-FE-Integration-Contracts.pdf	API client · auth header · error mapping · RBAC/menu gating ฝั่ง FE
ต้องจบก่อน (ลำดับงาน)	LLDD-FE-Integration-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Screen / Functional Scope

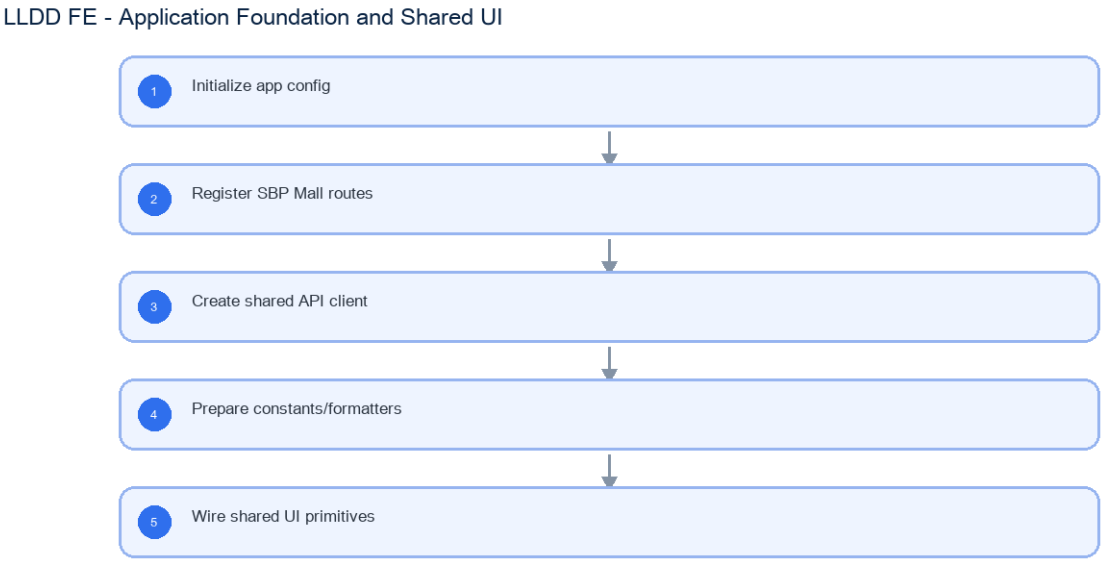
- Non-screen technical foundation
- Route/module registry เฉพาะ SBP Mall
- API client และ response typing
- Shared constants/menu/status mapping
- Mock data mapping
- CSS/tokens สำหรับ table/form/modal/responsive

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow & Sequence Diagram (Reference)

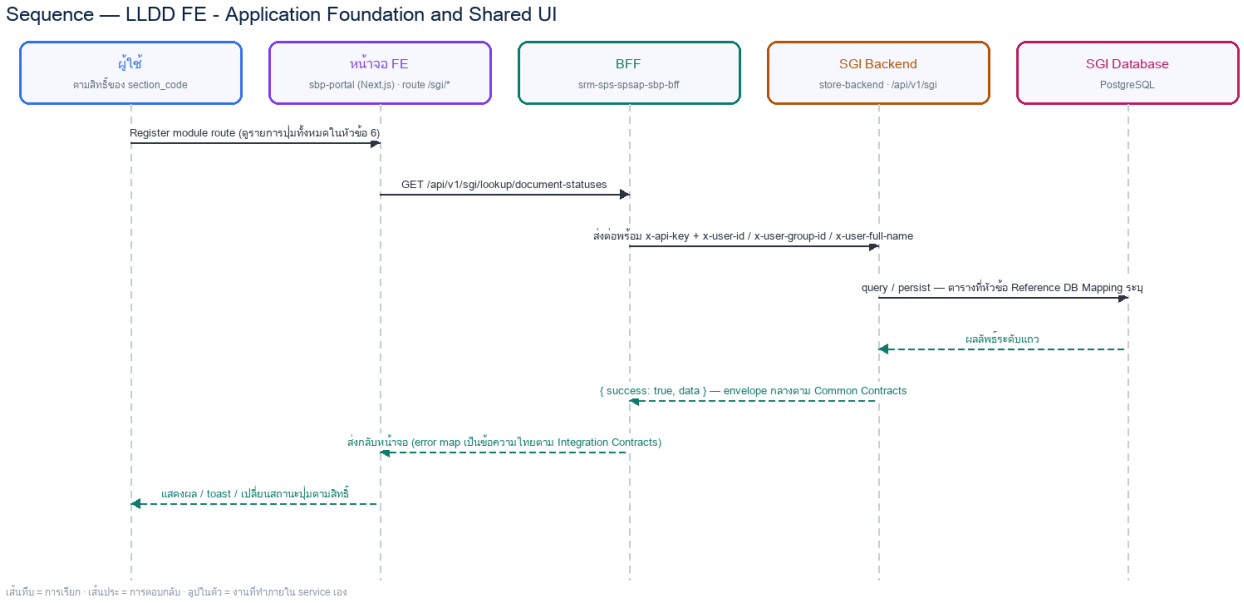
4.1 Implementation Flow (ลำดับขั้นการทำงาน)



รูปที่ 1: Implementation flow reference: LLDD FE - Application Foundation and Shared UI

4.2 Sequence Diagram (ใครคุยกับใคร ลำดับไหน)

ผู้แสดงและลำดับข้อความในภาพนี้สร้างจาก endpoint ในหัวข้อ 7 และตารางในหัวข้อ Reference DB Mapping ของเอกสารฉบับนี้เอง จึงตรงกับสัญญาเสมอ



รูปที่ 2: Sequence diagram: LLDD FE - Application Foundation and Shared UI

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
routePath	string	required	ต้อง map กับเมนู SBP Mall
apiBaseUrl	URL	required by env	ใช้กับทุก API call
statusCode	string	must map to status dictionary	ใช้ร่วมกับ StatusBadge
mockData	JSON	schema compatible with API response	ใช้ก่อน BE พร้อม

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	GET /api/v1/sgi/lookup/document-statuses
Progress	Initialize app config; Register SBP Mall routes; Create shared API client; Prepare constants/formatters
Output	ไม่มีตารางที่เอกสารนี้เขียนเอง — output คือ response ตาม envelope กลาง `{success, data}` และร่องรอยที่ตรวจย้อนได้ (log / sgi_consideration_logs / workflow_history ของ engine)

5.90 Application Foundation and Shared UI Component Contract

ID	Component / Scope	Single responsibility	Definition of done
C01	Non-screen technical foundation	ประกอบ app bootstrap, environment validation, providers และ error boundary โดยไม่สร้าง business screen	เปิด application shell ได้เมื่อ config ครบ และ fail-fast พร้อมข้อความเมื่อ config ขาด
C02	Route/module registry เฉพาะ SBP Mall	ลงทะเบียน route/module ของ SBP Mall และเชื่อม route guard กับ menuCode จาก API	ทุก route เข้าได้เฉพาะเมื่อ menu contract อนุญาตและ unknown route ไป not-found
C03	API client และ response typing	จัดโครงสร้าง DTO, API adapter และ query key กลางให้ response typing ตรงกับ contract	TypeScript build ผ่านและ feature ไม่ cast unknown response แบบ ad hoc
C04	Shared constants/menu/status mapping	รวม status/menu/action constants และ label resolver โดยให้ API dictionary เป็น source of truth	unknown code แสดง fallback ที่ trace ได้และไม่เพิ่มสถานะเองใน component
C05	Mock data mapping	สร้าง fixture/mock ให้ใช้ schema เดียวกับ response จริง รวม success, empty และ error	สลับ mock/real adapter ได้โดยไม่แก้ component props หรือ table mapping
C06	CSS/tokens สำหรับ table/form/modal/responsive	กำหนด token และ shared UI สำหรับ table, form, modal, badge และ responsive breakpoints	shared component ใช้งานได้บน desktop/tablet/mobile โดยข้อความและ control ไม่ล้น

5.91 Application Foundation and Shared UI API Adapter Map

Endpoint	Typed adapter purpose	Invoked by
GET /api/v1/sgi/lookup/document-statuses	โหลดสถานะเอกสารสำหรับ dropdown/badge	Register module route (bootstrap)

5.92 Application Foundation and Shared UI Interaction State Machine

Action	Trigger	API / State transition	Expected visible result
Register module route	bootstrap	client router	route guard รู้จักหน้า SBP Mall
Call API	React Query hook	shared API client	standard loading/error handling

5.93 Application Foundation and Shared UI Feature Failure Checks

Case	Feature-specific scenario	Expected evidence
FE-01	route registration	ไม่มี screenshot หรือ dashboard behavior ในเอกสารนี้
FE-02	API base missing	ทุก route ถูก register ผ่าน module registry
FE-03	status unknown	API error shape ใช้ร่วมกัน
FE-04	mock response compatible	ไม่มี dependency กับ Login/Auth ใหม่

Case	Feature-specific scenario	Expected evidence
FE-05	shared formatter output	CSS responsive base พร้อม

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Register module route	bootstrap	client router	route guard รู้จักหน้า SBP Mall
Call API	React Query hook	shared API client	standard loading/error handling

7. API Contract

GET /api/v1/sgi/lookup/document-statuses

โหลดสถานะเอกสารสำหรับ dropdown/badge

Query Params

```
{}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
-	none	No	No fields

Response

```
{
  "items": [
    {
      "statusCode": "06",
      "currentOwner": "somchai.k",
      "statusName": "รอฝ่าย SBP DSA ดำเนินการ",
      "code": "06",
      "label": "รอฝ่าย SBP DSA ดำเนินการ"
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].statusCode	string	Yes	canonical code; do not replace with display label
items[].currentOwner	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].statusName	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].code	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].label	string	Yes	UTF-8; use value domain described by endpoint purpose

8. Skeleton Code (โครงโค้ดตั้งต้นของหน้าจอนี้)

โค้ดชุดนี้อิง convention ของ portal เดิม `srm-sps-spsap-web-frontend` (build target `sbpm`): Next.js App Router + `'use client'`, PrimeReact ที่ห่อไว้แล้วใน `@/components/Form` และ `@/components/Table`, react-hook-form + yup, Zustand `permissionStore`, axios instance กลาง `@/lib/apiClient` และ react-query 5 — โปรเจกต์ไม่มี chart library จึงไม่มีโค้ดกราฟในเอกสารนี้ คัดลอกไปตั้งต้นได้ทันที แล้วเติมจุดที่กำกับ `TODO:`

8.1 ผังไฟล์ที่ต้องสร้าง

โครงไฟล์อิง portal เดิม (`srm-sps-spsap-web-frontend`, target `sbpm`) — โมดูล SGI อยู่ใต้ `src/app/(main)/sgi/*` และ import ผ่าน alias `@/*` ทุกจุด

Path ไฟล์	หน้าที่
src/app/(main)/sgi/layout.tsx	layout ของโมดูล + prefetch lookup (ไม่สร้าง QueryClient ใหม่)
src/constants/sgi/routes.ts	route registry ของโมดูล (ใช้ร่วมกับ url ที่มาจาก GET /menus)
src/services/sgi/lookup.service.ts	service — เรียก BFF ผ่าน apiClient (GET)
src/hooks/sgi/lookup.query.ts	hook — query key factory + useQuery/useMutation + invalidate
src/types/sgi/lookup.ts	types — request/response ตาม API contract ของเอกสารนี้

8.2 layout.tsx + route registry ของโมดูล SGI

```
'use client';
// src/app/(main)/sgi/layout.tsx — โครง layout ของโมดูล SGI
// (main)/layout.tsx เดิมมี AppHeader / AppSider / LottieLoader / QueryClientProvider อยู่แล้ว
// โมดูลนี้จึง "ห้าม" สร้าง QueryClient ใหม่ และ "ห้าม" สร้าง axios instance ของตัวเอง

import { ReactNode } from 'react';
import { useSgiLookupDocumentStatusesQuery } from '@/hooks/sgi/lookup.query';

/** route registry ของโมดูล — ใช้ประกอบลิงก์ภายใน ส่วนเมนู/สิทธิ์ยังมาจาก GET /menus เท่านั้น */
export const SGI_ROUTES = {
  waiting: '/sgi/documents/waiting',
  related: '/sgi/documents/related',
  create: '/sgi/documents/create',
  // path คงที่ + query — `output: export` ไม่รองรับ dynamic segment ตาม runtime
  detail: (docNo: string) => `/sgi/documents/detail/?docNo=${encodeURIComponent(docNo)}`,
  report: '/sgi/report/status-summary',
  master: {
    factors: '/sgi/master/factors',
    competitors: '/sgi/master/competitors',
  },
} as const;

/**
 * ● ราก path ทุกตัวต้องมี `page.tsx` ของตัวเอง (เพิ่ม 2026-09-23)
 *
 * `output: "export"` ไม่มี Node ตอน runtime -> Next **ไม่ apply `redirects()` ใน
 * next.config ให้เลย** ราก path ที่มีแต่โฟลเดอร์ลูกจึงขึ้น **404** ทันที
 * เจอจริง: `/sgi/` ขึ้น 404 ทั้งที่ **breadcrumb ของทุกหน้าในโมดูลลิงก์มาที่นี่**
 *
 * ทางแก้คือหน้า redirect ฝั่ง client ตัวเล็ก ๆ — อ่านปลายทางจาก registry นี้
 * ใช้ `router.replace` ไม่ใช่ `push` (กันปุ่ม Back วนกลับมาที่ทางผ่าน)
 * คง query string เดิมไว้ และมีข้อความระหว่างรอ ไม่ปล่อยจอว่าง
 *
 * | ราก | ปลายทาง |
 * |---|---|
 * | `/sgi/` | waiting |
 * | `/sgi/documents/` | waiting |
 * | `/sgi/master/` | master.factors (รายการแรกของกลุ่มบนเมนู) |
 * | `/sgi/report/` | report (กลุ่มนี้มีรายงานเดียว — ถ้าวันหนึ่งมีหลายตัว ให้ทำเป็นสารบัญแทน redirect) |
 */

export default function SgiLayout({ children }: { children: ReactNode }) {
  // prefetch lookup ที่ทุกหน้าในโมดูลใช้ร่วมกัน (master -> staleTime ยาว)
  useSgiLookupDocumentStatusesQuery();

  // TODO: ใส่ ErrorBoundary ของโมดูล และ empty state เมื่อ permission ยังโหลดไม่เสร็จ
  return <div className="sgi-module">{children}</div>;
}
```

8.3 service — `src/services/sgi/lookup.service.ts`

▲ `src/services/sgi/lookup.service.ts` เป็น ไฟล์ร่วมของโมดูล SGI (เอกสาร FE หลายฉบับที่ใช้ domain `lookup` ประกาศไฟล์นี้เหมือนกัน) — เวลา implement ให้ merge เพิ่ม เข้าไฟล์เดิม ห้ามเขียนทับทั้งไฟล์ มิฉะนั้น type/function ของเอกสารฉบับก่อนหน้าจะหายไปเงียบ ๆ

```
// src/services/sgi/lookup.service.ts
// apiClient = axios instance กลาง (baseURL = bffUrl ซึ่งรวม /api/v1 แล้ว, withCredentials, refresh-token interceptor, global loading)
// ห้ามสร้าง axios instance ใหม่ และห้าม set Authorization header เอง — session อยู่ใน httpOnly cookie ของ BFF

import apiClient from '@/lib/apiClient';
import type { ApiResponse } from '@/types/sgi/common';
import type * as T from '@/types/sgi/lookup';

/** GET /api/v1/sgi/lookup/document-statuses — โหลดสถานะเอกสารสำหรับ dropdown/badge */
export async function getSgiLookupDocumentStatuses(): Promise<T.SgiLookupDocumentStatusesResponse> {
  const { data } = await apiClient.get<ApiResponse<T.SgiLookupDocumentStatusesResponse>>('/sgi/lookup/document-statuses');
  return data.data;
}

// ── envelope: อ่าน `data.data` ชั้นเดียว (ยืนยันจากโค้ดจริงของ BFF 2026-09-04) ──
// BFF มี ResponseInterceptor ระดับ global (src/common/interceptors/response.interceptor.ts)
// ที่ห่อผลลัพธ์ของ controller เป็น { success, data, requestId } ให้เสมอ
// ▲ ถ้า client service ของ BFF คืน `response.data` ดิบ (= envelope ของ store-backend)
// interceptor จะเห็นคีย์ success แล้วห่อซ้ำ → FE ได้ { success, data: { data: <payload> } }
// สัญญาที่ตกลง: **BFF ต้อง unwrap ของ store-backend ก่อน 1 ชั้น** (คืน response.data.data)
// FE จึงอ่าน data.data ชั้นเดียวตามโค้ดด้านบน · requestId ใช้อ้างอิงตอนแจ้งปัญหา
```

8.4 types — `src/types/sgi/lookup.ts`

▲ `src/types/sgi/lookup.ts` เป็น ไฟล์ร่วมของโมดูล SGI (เอกสาร FE หลายฉบับที่ใช้ domain `lookup` ประกาศไฟล์นี้เหมือนกัน) — เวลา implement ให้ merge เพิ่ม เข้าไฟล์เดิม ห้ามเขียนทับทั้งไฟล์ มิฉะนั้น type/function ของเอกสารฉบับก่อนหน้าจะหายไปเงียบ ๆ

```
// src/types/sgi/lookup.ts — ตรงกับตาราง API ในเอกสารนี้
// วันที่/เดือนเป็น ค.ศ. ทั้ง payload (ISO) และ display — ไม่แปลงเป็น พ.ศ. (มติ 2026-08-06)

/** GET /api/v1/sgi/lookup/document-statuses — 1 แถวในตาราง */
export interface SgiLookupDocumentStatusesItem {
  statusCode: string;
  currentOwner: string;
  statusName: string;
  code: string;
  label: string;
}
export interface SgiLookupDocumentStatusesResponse { items: SgiLookupDocumentStatusesItem[]; }

// TODO: ใส่ nullable / required ให้ตรงกับ contract ฉบับล่าสุดของ BE
```

8.5 react-query keys + hooks — `src/hooks/sgi/lookup.query.ts`

▲ `src/hooks/sgi/lookup.query.ts` เป็น ไฟล์ร่วมของโมดูล SGI (เอกสาร FE หลายฉบับที่ใช้ domain `lookup` ประกาศไฟล์นี้เหมือนกัน) — เวลา implement ให้ merge เพิ่ม เข้าไฟล์เดิม ห้ามเขียนทับทั้งไฟล์ มิฉะนั้น type/function ของเอกสารฉบับก่อนหน้าจะหายไปเงียบ ๆ

```
// src/hooks/sgi/lookup.query.ts
import { useQuery } from '@tanstack/react-query';
import * as api from '@/services/sgi/lookup.service';
import type * as T from '@/types/sgi/lookup';

export const lookupKeys = {
  all: ['sgi', 'lookup'] as const,
  sgiLookupDocumentStatuses: () => [...lookupKeys.all, 'sgiLookupDocumentStatuses'] as const,
};

export function useSgiLookupDocumentStatusesQuery() {
  return useQuery({
    queryKey: lookupKeys.sgiLookupDocumentStatuses(),
    queryFn: () => api.getSgiLookupDocumentStatuses(),
    staleTime: 30_000, // TODO: ปรับตามความถี่ของข้อมูลหน้านี้
  });
}
```

- ทุกหน้าเช็คสิทธิ์ด้วย `permissionStore.hasPermission(url, 'canView'|'canManage'|'canExport'|'canOther')` แล้ว render `<AccessDenied />` เมื่อไม่มีสิทธิ์
- เมนู/สิทธิ์มาจาก `GET /menus` และ `GET /groups/current-user/permissions` — ห้าม hardcode role หรือรายการเมนูใน FE

- session อยู่ใน httpOnly cookie ของ BFF (`withCredentials: true`) — FE ไม่เก็บและไม่แนบ token เอง
- payload และการแสดงผลใช้วันที่ ค.ศ. เสมอ ผ่าน formatter กลางจุดเดียว — ไม่แปลงเป็น พ.ศ. (มติ 2026-08-06)
- ข้อความ error แสดงจาก `error.message` ของ BE ตรง ๆ (ห้าม paraphrase) — fallback ใช้เฉพาะกรณี network error

9. Processing Flow

Step	Description
1	Initialize app config
2	Register SBP Mall routes
3	Create shared API client
4	Prepare constants/formatters
5	Wire shared UI primitives

10. Acceptance Criteria

- ไม่มี screenshot หรือ dashboard behavior ในเอกสารนี้
- ทุก route ถูก register ผ่าน module registry
- API error shape ใช้ร่วมกัน
- ไม่มี dependency กับ Login/Auth ใหม่
- CSS responsive base พร้อม

11. Developer Test Checklist

No	Test
1	route registration
2	API base missing
3	status unknown
4	mock response compatible
5	shared formatter output

12. Unit Test Scope

7 ชั่วโมง (25% ของ implementation 28 ชั่วโมง) · เครื่องมือ: Jest + React Testing Library + msw (mock API layer)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
`routePath`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required · รูปแบบ: string
`apiBaseUrl`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required by env · รูปแบบ: URL
`statusCode`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: must map to status dictionary · รูปแบบ: string
`mockData`	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: schema compatible with API response · รูปแบบ: JSON
business rule	logic	ไม่มี screenshot หรือ dashboard behavior ในเอกสารนี้

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
business rule	logic	ทุก route ถูก register ผ่าน module registry
business rule	logic	API error shape ใช้ร่วมกัน
business rule	logic	ไม่มี dependency กับ Login/Auth ใหม่
business rule	logic	CSS responsive base พร้อม
`GET /api/v1/sgi/lookup/document-statuses`	api client	hook/service เรียกเส้นนี้ด้วยพารามิเตอร์ถูกต้อง · map {success:true,data} เป็น state ที่หน้าจอใช้ · เจอ {success:false,error} แล้วแสดงข้อความไทย verbatim (mock ด้วย msw)
component	render	render ด้วย React Testing Library แล้วเห็น element ตาม field/action contract ของเอกสารนี้
hook/state	interaction	ยิง action แล้ว state เปลี่ยนตามที่ระบุ และเรียก API layer ที่ mock ไว้ด้วยพารามิเตอร์ถูกต้อง
error path	ui	API ตอบ error envelope แล้วหน้าจอต้องแสดงข้อความไทย verbatim ไม่ crash

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด